

Prototype Design Pattern in C# with Examples

In this article, I will discuss the **Prototype Design Pattern in C#** with Examples. Please read our previous article discussing the [Fluent Interface Design Pattern in C#](#) with Examples. The Prototype Design Pattern falls under the category of the Creational Design Pattern. At the end of this article, you will understand the following pointers:

1. [What is the Prototype Design Pattern?](#)
2. [What is Object Cloning in C#?](#)
3. [Example to Understand Object Cloning in C#.](#)
4. [Prototype Design Pattern UML \(Class\) Diagram.](#)
5. [When to Use Prototype Design Pattern in C#?](#)

What is the Prototype Design Pattern?

According to the GoF Definition, the **Prototype Design Pattern specifies the kind of objects to create using a prototypical instance and creates new objects by copying this prototype.** The Prototype Design Pattern creates objects by copying an existing object, known as the prototype.

To simplify the above definition, we can say that the Prototype Design Pattern gives us a way to create new or cloned objects from the existing object of a class. That means it clones the existing object with its data into a new object. If we make any changes to the cloned object (i.e., new object), it does not affect the original object.

This pattern is useful when the creation of an object is time-consuming or complex in terms of resources. Instead of going through the costly process of creating a fresh object from scratch, an existing instance serves as a prototype, and new objects can be created by copying this prototype.

Example to Understand Prototype Design Pattern in C#:

Let us understand the Prototype Design Pattern in C# with an example. In C#, when we try to copy one object to another using the assignment (=) operator, both objects share the same memory address. The reason is the assignment operator (=) copies the reference, not the object, except when there is a value type field. This operator will always copy the reference, not the actual object, when working with the Reference type. Please have a look at the following image for a better understanding.

```
public class Employee
{
    public string Name { get; set; }
    public string Department { get; set; }
}
```

```
static void Main(string[] args)
{
    Employee emp1 = new Employee();
    emp1.Name = "Anurag";
    emp1.Department = "IT";

    Employee emp2 = emp1;
    emp2.Name = "Pranaya";
}
```

Here, we create emp2 object from emp1.

Then we change the name of emp2 but this will also change the name of emp1.

This is because when we are assigning one object to another object, it is by default call by ref.

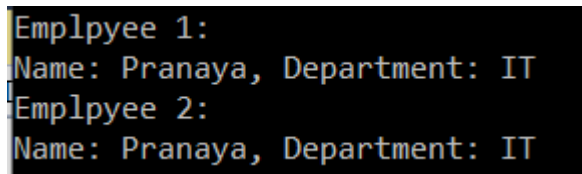
As you can see in the above image, we have one class called Employee, having two properties, i.e., Name and Department. Within the Main method, first, we create an instance of the Employee class (i.e., emp1) and set values for Name and Department properties. Then, we create another employee instance (i.e., emp2) by assigning the existing employee object (i.e., emp1). When we change the Name property value of the emp2 object, it also changes the name of the emp1 object, and this is because both emp1 and emp2 share the same memory. So, remember that when we use the assignment (=) operator, it is Call-By-Reference only. The Complete Examples Code is given below.

```

using System;
namespace PrototypeDesignPattern
{
    class Program
    {
        static void Main(string[] args)
        {
            // Creating an Instance of Employee Class
            Employee emp1 = new Employee();
            emp1.Name = "Anurag";
            emp1.Department = "IT";
            // Creating another Instance of Employee with Existing Employee
            Instance using Assignment Operator
            // Both emp1 and emp2 share the same memory location as =
            Operator uses Call By Reference Mechanism
            Employee emp2 = emp1;
            // Changing the Name Property Value of emp2 instance,
            // it will also change the Name Property Value of emp1 instance
            emp2.Name = "Pranaya";
            Console.WriteLine("Employee 1: ");
            Console.WriteLine("Name: " + emp1.Name + ", Department: " +
emp1.Department);
            Console.WriteLine("Employee 2: ");
            Console.WriteLine("Name: " + emp2.Name + ", Department: " +
emp2.Department);
            Console.Read();
        }
    }
    public class Employee
    {
        public string Name { get; set; }
        public string Department { get; set; }
    }
}

```

Output:



```

Employee 1:
Name: Pranaya, Department: IT
Employee 2:
Name: Pranaya, Department: IT

```

What is Object Cloning in C#?

When we talk about Object Cloning in C#, it means it is all about the Call By Value. So, if we change one object, it will not affect the other. Let us see how to clone the object to another object. To do so, C# provides one method called **MemberwiseClone**, which will create a new complete copy of the object with a different memory. For a better understanding, please have a look at the below image.

```

public class Employee
{
    public string Name { get; set; }
    public string Department { get; set; }

    public Employee GetClone()
    {
        return (Employee)this.MemberwiseClone();
    }
}

```

The **MemberwiseClone** method creates a brand new copy of the object it cloned.



Client creates the object of Employee class

```

static void Main(string[] args)
{
    Employee emp1 = new Employee();

    emp1.Name = "Anurag";
    emp1.Department = "IT";

    Employee emp2 = emp1.GetClone();

    emp2.Name = "Pranaya";
}

```

The First copy of the Employee object is created

Values are set for the first object

Second Copy of the object is created by calling the GetClone method

Changing the value in second object

As shown in the above image, within the Employee class, we created one method, i.e., GetClone. As part of that method, we return a clone object using the **MemberwiseClone** method. Then, from the client code, we create a new instance of the Employee class and assign the properties with some values. Next, we create the second object by calling the GetClone method, which returns a new complete copy of the emp1 object. Now, both the objects (i.e., emp1 and emp2) are independent, and if we make any changes to any object, it will not affect other objects.

Note: Remember that the MemberwiseClone() method creates a shallow copy of the object. If your object contains other objects and you want a deep copy where even those internal objects are duplicated, you'd need to implement custom logic in the Clone() method. In our next article, we will discuss [shallow and deep copies](#) in detail.

Example to Understand Object Cloning in C#

The following code example shows how to create a Duplicate or Cloned Object from an Existing object using Object Cloning.

```

using System;
namespace PrototypeDesignPattern
{
    class Program
    {
        static void Main(string[] args)
        {
            // Creating an Instance of Employee Class
            Employee emp1 = new Employee();
            emp1.Name = "Anurag";
            emp1.Department = "IT";
            // Instead of using Assignment Operator, now use the Clone
            method to create a Cloned Object
            // Both emp1 and emp2 having different Memory Address

```

```

        Employee emp2 = emp1.GetClone();
        // Changing the Name Property Value of emp2 instance,
        // will not change the Name Property Value of emp1 instance
        emp2.Name = "Pranaya";
        Console.WriteLine("Employee 1: ");
        Console.WriteLine("Name: " + emp1.Name + ", Department: " +
emp1.Department);
        Console.WriteLine("Employee 2: ");
        Console.WriteLine("Name: " + emp2.Name + ", Department: " +
emp2.Department);
        Console.Read();
    }
}
public class Employee
{
    public string Name { get; set; }
    public string Department { get; set; }
    public Employee GetClone()
    {
        // MemberwiseClone Method Creates a shallow copy of the current
System.Object
        // So typecast the Object Appropriate Type
        // In this case, typecast to Employee type
        return (Employee)this.MemberwiseClone();
    }
}
}

```

Output:

```

Employee 1:
Name: Anurag, Department: IT
Employee 2:
Name: Pranaya, Department: IT

```

Points to Remember about the MemberwiseClone Method in C#:

1. The MemberwiseClone method is part of the **System.Object** class and creates a shallow copy of the given object.
2. MemberwiseClone Method only copies the non-static fields of the object to the new object.
3. In copying, if a field is a value type, a bit-by-bit copy of the field is performed. If a field is a reference type, the reference is copied, but the referenced object is not.

So, once we have understood Object Cloning, let's proceed to understand how to implement Prototype Design Patterns in C# using Object Cloning.

Real-Time Example to Understand Prototype Design Patterns in C#:

As we already discussed, the Prototype Design Pattern in C# is used when we want to create a Duplicate or Cloned Object from an Existing object to enhance the application's performance. That means we need to use the Prototype Design Pattern when creating an object is time-consuming, costly, and complex.

Let us proceed and implement the Prototype Design Pattern in C# step by step. Once we implement the Prototype Design Pattern, we will see the UML Diagram and compare the Example with the Prototype Design Pattern UML (or

Class) Diagram. So, first, create a class file named **Employee.cs** and then copy and paste the following code into it. The following class is the Prototype Abstract Class, which is used for the types of objects that can be cloned.

```
namespace PrototypeDesignPattern
{
    //The Prototype Abstract Class
    //This is an abstract class which is used for the types of object that
    can be cloned itself.
    public abstract class Employee
    {
        public string Name { get; set; }
        public string Department { get; set; }
        public string Type { get; set; }
        public abstract Employee GetClone();
        public abstract void ShowDetails();
    }
}
```

As you can see, the above class has few properties common with the next-level subclasses and two abstract methods that the child classes will implement.

PermanentEmployee.cs

So, next, create a class file with the name **PermanentEmployee.cs** and then copy and paste the following code into it. Here, you can see that the class implements the Employee abstract class and provides implementations for the two abstract methods. In the GetClone method, we have written the logic to create a shallow copy of the current object. In our next article, we will discuss Shall Copy and Deep Copy in detail.

```
using System;
namespace PrototypeDesignPattern
{
    //This is a class that implements the Prototype interface for cloning
    itself.
    public class PermanentEmployee : Employee
    {
        public int Salary { get; set; }
        public override Employee GetClone()
        {
            // MemberwiseClone Method Creates a shallow copy of the current
            System.Object
            // So typecast the Object Appropriate Type
            // In this case, typecast to PermanentEmployee type
            return (PermanentEmployee)this.MemberwiseClone();
        }
        public override void ShowDetails()
        {
            Console.WriteLine("Permanent Employee");
            Console.WriteLine($" Name:{this.Name}, Department:
{this.Department}, Type:{this.Type}, Salary: {this.Salary}\n");
        }
    }
}
```

TemporaryEmployee.cs

Next, create a class file named **TemporaryEmployee.cs** and copy and paste the following code. Here, you can see the class implements the Employee abstract class and provides implementations for the two abstract methods. In the GetClone method, we have written the logic to create a shallow copy of the current object. The following code is self-explained, so please go through the comment lines for a better understanding.

```
using System;
namespace PrototypeDesignPattern
{
    //This is a class that implements the Prototype interface for cloning
    itself.
    public class TemporaryEmployee : Employee
    {
        public int FixedAmount { get; set; }
        public override Employee GetClone()
        {
            // MemberwiseClone Method Creates a shallow copy of the current
            System.Object
            // So typecast the Object Appropriate Type
            // In this case, typecast to TemporaryEmployee type
            return (TemporaryEmployee)this.MemberwiseClone();
        }
        public override void ShowDetails()
        {
            Console.WriteLine("Temporary Employee");
            Console.WriteLine($" Name:{this.Name}, Department:
{this.Department}, Type:{this.Type}, FixedAmount: {this.FixedAmount}\n");
        }
    }
}
```

We need to test whether the Clone Method works as expected within the Client code. In our example, the Main method of the Program class is nothing but the client. So, please modify the Main method of the Program class as follows.

```
using System;
namespace PrototypeDesignPattern
{
    class Program
    {
        static void Main(string[] args)
        {
            // Creating an Instance of Permanent Employee Class
            Employee emp1 = new PermanentEmployee()
            {
                Name = "Anurag",
                Department = "IT",
                Type = "Permanent",
                Salary = 100000
            };
            //Creating a Clone of the above Permanent Employee
            Employee emp2 = emp1.GetClone();
            // Changing the Name and Department Property Value of emp2
            instance,
            // will not change the Name and Department Property Value of
            the emp1 instance
        }
    }
}
```

```

emp2.Name = "Pranaya";
emp2.Department = "HR";
emp1.ShowDetails();
emp2.ShowDetails();
// Creating an Instance of Temporary Employee Class
Employee emp3 = new TemporaryEmployee()
{
    Name = "Preety",
    Department = "HR",
    Type = "Temporary",
    FixedAmount = 200000
};
//Creating a Clone of the above Temporary Employee
Employee emp4 = emp3.GetClone();
// Changing the Name and Department Property Value of emp4
instance,
    // will not change the Name and Department Property Value of
the emp3 instance
emp4.Name = "Priyanka";
emp4.Department = "Sales";
emp3.ShowDetails();
emp4.ShowDetails();

Console.Read();
}
}
}

```

Output:

```

Permanent Employee
Name:Anurag, Department: IT, Type:Permanent, Salary: 100000

Permanent Employee
Name:Pranaya, Department: HR, Type:Permanent, Salary: 100000

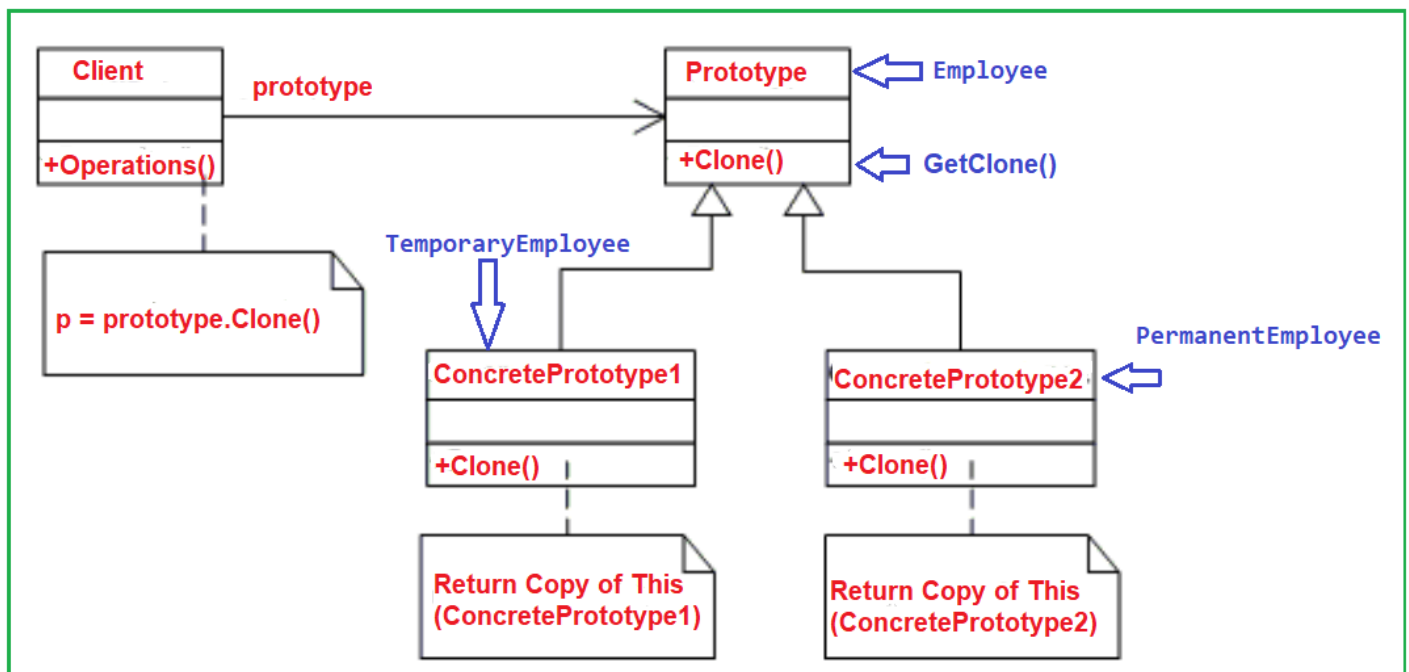
Temporary Employee
Name:Preety, Department: HR, Type:Temporary, FixedAmount: 200000

Temporary Employee
Name:Priyanka, Department: Sales, Type:Temporary, FixedAmount: 200000

```

Prototype Design Pattern UML (Class) Diagram:

Once we understand how to implement the Prototype Design Pattern in C#, let us try to understand its UML (Unified Modeling Language) or Class diagram. For a better understanding, please look at the following diagram, which shows the different components of the Prototype Design Pattern. Here, I am comparing the Prototype Design Pattern UML diagram with our example so that you can easily understand the different components of the Prototype Design Pattern.



Let us understand each component of the Prototype Design Pattern:

1. **Prototype:** This will be an interface or abstract class used for the types of objects that can be cloned. In our example, it is going to be the Employee Abstract Class.
2. **ConcretePrototype:** This class will implement the Prototype abstract class or interface for cloning. In our example, it will be the PermanentEmployee and TemporaryEmployee Classes.
3. **Client:** The client is the class that creates a new object by asking a prototype to clone itself.

When to Use Prototype Design Pattern in C#?

The Prototype Design Pattern is useful in real-time applications where performance, efficiency, and flexibility in object creation are critical. The following are some of the scenarios where the use of the Prototype Design Pattern can be effective:

- **High-Cost Initialization:** If initializing an object is resource-intensive, possibly because it requires complex computation, database operations, or reading from external files, using a prototype can be beneficial. Once the initial object is created, subsequent objects can be cloned, significantly reducing the cost of creating each new instance.
- **Object Pooling:** Real-time systems often use object pools (pre-instantiated objects ready to be reused) to manage resources efficiently, especially in high-load scenarios. The Prototype pattern can facilitate the management of such pools, allowing for the resetting and reusing of objects by cloning a clean prototype instead of recreating objects from scratch.
- **When Flexibility is Required Over Static Behavior:** Prototypes provide more flexibility in how objects can be constructed compared to other creational patterns like Factory or Builder, which might require new classes to be created for each new object type. With prototypes, you can work with object instances that can be cloned to produce new instances, facilitating dynamic behavior.
- **Managing Families of Related Objects:** If you need to manage families of related objects designed to be used together and need to ensure that the objects you create are always in a particular state or configuration, prototypes can provide a predefined set of objects that are already configured for use.

Shallow vs. Deep Copy:

Shallow Copy: The MemberwiseClone method in C# performs a shallow copy. It copies the values of the fields of an object to a new object. If the field is a value type, a bit-by-bit copy of the field is performed. For reference types, the reference is copied, not the object itself.

Deep Copy: If your object has reference-type fields, you might need to implement deep cloning to avoid shared references in your cloned objects. Deep cloning involves creating copies of the objects referenced by the fields.

While working with object cloning, we need to understand two things: Shallow Copy and Deep Copy. So, in the next article, I will discuss [Shallow Copy and Deep Copy in C#](#) with Examples. In this article, I try to explain the **Prototype Design Pattern in C#** with Examples. I hope you understand the need and use of the Prototype Design Pattern in C# with Examples.